Agent Postmortem Generator

Part 7 - Rendering & Export Engine

Deterministically render the finalized postmortem into stable human- and machine-readable artifacts on the local filesystem.

Project Name	Agent Postmortem Generator
Part Number	7
Module Name	Rendering & Export Engine
Document Type	PDF-ready engineering design document

Version: MVP design draft

Executive Overview

The Rendering & Export Engine converts the finalized postmortem artifact into concrete output files. It does not decide what the report says, what the analysis means, or which evidence matters. Those responsibilities stay upstream. This module only takes the structured report object produced by the Postmortem Generation Engine, plus any already-derived graph artifact, and turns them into stable files a developer can inspect, diff, attach to a bug report, or check into a repository.

The module exists because the project needs a clean boundary between meaning and presentation. Once the postmortem has been assembled, the rest of the system should stop thinking in terms of domain objects and start thinking in terms of portable artifacts. That boundary is what keeps the project testable, local-first, and maintainable. The output layer is deliberately boring. Boring is good. Boring survives contact with real users.

This module sits at the end of the pipeline. It consumes a report bundle that is already complete and validated, and emits Markdown, JSON, and Mermaid source artifacts using deterministic naming and atomic file writes. Future PDF support belongs here eventually, but not in the MVP path.

Format	Owned in MVP	Purpose	Notes
Markdown report	Yes	Primary human-readable output	Stable section order, LF line endings, GitHub-friendly formatting.
JSON report	Yes	Machine-readable artifact for automation and tests	Canonical key order and no presentation-only content.
Mermaid graph source	Yes, when graph payload exists	Portable graph representation	Rendered from graph artifact already produced upstream.
PDF	No	Later presentation export	Deferred to avoid layout complexity in the MVP.

Problem Statement

The project needs a deterministic way to convert the final report model into artifacts that humans and machines can consume without re-running analysis or depending on any remote system. The hard part is not writing files. The hard part is preserving meaning while changing representation. The report is structured, evidence-based, and tightly ordered by the analysis engine. The export layer must preserve that structure exactly, while also producing output that is readable, stable across platforms, and suitable for regression tests.

This is harder than it looks because output formats have different failure modes. Markdown can silently drift if escaping or ordering is sloppy. JSON can become unreadable or unstable if key

order or whitespace varies. Mermaid can break when node identifiers are not sanitized consistently. Filesystems can partially write data or produce inconsistent results when a process crashes. The module must deal with all of that while remaining small enough for a 2 to 3 engineer team to reason about.

The MVP needs this module because the rest of the system is useless if it cannot produce something the user can actually open. Engineers do not want a thesis about failure reconstruction. They want a report they can diff, inspect, attach to an issue, and trust. This module is the point where the project becomes tangible.

Goals

- Render the finalized report into stable Markdown and JSON artifacts without changing the underlying meaning.
- Emit Mermaid graph source when a graph artifact has already been produced upstream.
- Guarantee deterministic file names, stable ordering, and reproducible formatting for identical inputs and configuration.
- Write files atomically so a failed export never leaves a half-written report behind.
- Keep the formatting layer local-only, dependency-light, and easy to regression test with golden files.

Non-Goals

- Do not generate analysis, infer causality, or decide what belongs in the report.
- Do not derive graph structure. The graph model comes from the Graph Generation Engine or upstream report bundle.
- Do not implement PDF layout in the MVP. That is a later presentation concern.
- Do not build a web viewer, collaborative editor, upload service, or remote artifact store.
- Do not let users inject arbitrary templates or scripts into the rendering pipeline.
- Do not support dozens of output formats just because expansion looks impressive in a roadmap slide.

Responsibilities

This module owns	This module must never own
Format-specific rendering for Markdown, JSON, and Mermaid source.	Failure analysis, root-cause selection, or report content generation.
Filename planning, output directory resolution, and artifact inventory creation.	Parsing raw traces or inspecting framework-specific execution objects.
Escaping, ordering, and serialization rules that keep outputs deterministic.	Business logic that changes what the report says.
Atomic file writes, overwrite policy, and export error propagation.	Network access, remote storage, or external publishing.
A stable export contract that downstream	A second opinion on the analysis engine or

tools can test against.	graph engine output.

Inputs

The module accepts a finalized report bundle, not raw traces and not intermediate framework objects. The bundle is assumed to be validated upstream. This module still performs structural checks because an export layer that blindly trusts its caller eventually becomes a landfill of mysterious file corruption.

Input	Required	Purpose	Validation rule
Postmortem report object	Yes	Main content source for Markdown and JSON	Must include the report sections, evidence references, summary metadata, and stable ordering metadata.
Execution graph artifact	Conditional	Source for Mermaid export	Required if graph export is requested; must already be derived by the graph module.
Export plan	Yes	Select targets, output path, overwrite behavior	Must resolve to a local filesystem path inside an allowed base directory.
Render options	Optional	Formatting preferences	Only a small, closed set of options is supported in MVP.

Validation rules are intentionally strict: unsupported target formats fail fast, unsafe output paths are rejected, and missing required payloads stop the export before any file is written. This module should not guess. If the caller requests a graph artifact without providing graph data, the correct response is an explicit error, not a placeholder diagram invented for optimism.

- Assume all narrative content is already finalized upstream.
- Assume section ordering is already defined by the report schema.
- Assume graph node and edge identities are stable before rendering begins.
- Assume the filesystem is local and writable unless validation proves otherwise.

Outputs

The module writes files to the requested output directory and returns a structured export result to the caller. The export result is useful for logs, tests, and downstream CLI summaries, but it is not a second source of truth. The files are the artifact. The result object is just the receipt.

Artifact	Format	Default filename	Purpose
Human-readable	Markdown	postmortem.md	Primary document

report			for reading, sharing, and version control diffs.
Structured report payload	JSON	postmortem.json	Machine-readable artifact for automation and regression tests.
Graph source	Mermaid	execution-graph.mmd	Portable graph text for editors, renderers, or later conversion.
Export result object	In-memory	Not persisted	Lists produced artifacts, byte sizes, and checksums.

PDF export is explicitly deferred. If and when it arrives, it should reuse the same source bundle and stable ordering rules rather than creating a parallel rendering path. The module should not grow a hidden second formatting stack just because someone wanted a prettier demo.

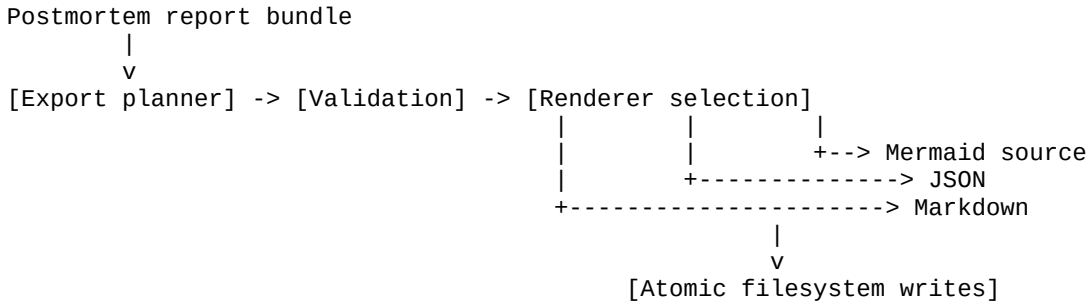
Internal Design

The design is a small pipeline with four explicit stages: plan, render, write, and verify. Each stage has one job and one failure surface. That keeps the module composable and keeps tests targeted. The implementation should be boring in the best possible way.

Main subcomponents:

- Export Planner: resolves output targets, output directory, file names, and overwrite policy.
- Markdown Renderer: converts the structured report into a stable GitHub-friendly Markdown document.
- JSON Serializer: emits the structured report as canonical JSON with deterministic key ordering.
- Mermaid Emitter: serializes the already-derived graph artifact into Mermaid source text.
- Atomic Writer: writes temporary files and renames them into place only after rendering succeeds.
- Consistency Guard: verifies that requested outputs and available source artifacts match before writing begins.

Workflow:



```
                    |
                    v
          ExportResult + file paths
```

Data flow: the module receives the final structured report, chooses the requested targets, renders each target from the same source bundle, and writes the outputs only if every requested target has been successfully rendered in memory. The files are emitted as the final step. There is no incremental mutation of the source model, no partial semantic rewrite, and no downstream interpretation of the rendered files.

Important internal decisions:

- Use templates only for presentation structure, not for business logic.
- Do not parse free-form prose from the report model; consume structured fields only.
- Keep line endings normalized to LF for portability and snapshot stability.
- Keep JSON serialization deterministic by using a fixed key order and stable array ordering.
- Keep Markdown sections in a fixed schema-defined order rather than allowing caller-defined shuffling.

Data Model

Entity	Key fields	Why it exists
ExportPlan	output_dir, targets, overwrite, filename_stem, format_options	Captures exactly what should be rendered and where it should go.
RenderedArtifact	kind, path, bytes_written, sha256, content_type	Represents one file emitted by the module.
ExportResult	success flag, artifacts list, warnings, error list	Provides a structured receipt for CLI output and tests.
ReportBundle	report sections, evidence, summary, graph payload	Source input from the generation layer. This module does not own it.

Relationship rules: one ExportPlan can produce many RenderedArtifact records; every RenderedArtifact belongs to exactly one export run; a ReportBundle can be rendered into multiple formats but is never mutated by the renderer. The model should be intentionally shallow. There is no need for a meta-framework of renderer graphs and adapter factories. The project already has enough architecture to impress people who never ship.

The data model exists to preserve a clean seam between meaning and representation. The report bundle defines what must be said. The export layer defines how it is turned into files. Mixing those concerns is what causes drift, especially when the report structure changes later.

Algorithms / Processing Logic

1. Resolve the export plan and validate the output directory, target set, and overwrite policy before any file is created.

2. Check that every requested output has its source payload available. For example, Mermaid export requires a graph artifact. If the artifact is missing, fail fast.
3. Render each requested artifact in memory using deterministic formatting rules and the shared report bundle.
4. Compute checksums and byte counts for the rendered artifacts before writing.
5. Write each file to a temporary path in the same directory, flush it, then rename it atomically into place.
6. Return a structured ExportResult with paths, sizes, checksums, and any non-fatal warnings.

Edge cases:

- If the output directory already contains files with the same name and overwrite is disabled, abort before rendering begins.
- If a filename stem contains path traversal sequences or unsupported characters, sanitize or reject it based on the configured policy. Do not silently rewrite it into something unpredictable.
- If the report contains characters that require escaping in Markdown or Mermaid, escape them deterministically rather than trying to be clever with heuristics.
- If a section is intentionally empty, render the upstream-provided empty-state text rather than inventing commentary.
- If a filesystem write fails midway, delete temporary files and surface a clear error that identifies the target artifact.

Deterministic behavior requirements:

- Identical inputs and configuration must produce identical artifact content, aside from any explicitly configured metadata fields.
- Artifact order must be fixed.
- List ordering must follow the source bundle order or a schema-defined order, never incidental map ordering.
- Whitespace, line endings, and code fence formatting must not vary by operating system.
- The module must not embed non-deterministic timestamps inside the primary content unless the report schema explicitly asks for them.

Tradeoffs

Tradeoff	Chosen design	Reason
Simplicity vs flexibility	Simple fixed renderers, not a plugin zoo	The MVP needs stable outputs more than arbitrary extension points.
Generality vs specificity	Specific support for Markdown, JSON, and Mermaid only	That covers the real use case without multiplying failure modes.
Determinism vs heuristic	Pure formatting from	The export layer should never

inference	structured data, no inference	become a second source of truth.
Atomicity vs partial success	All requested targets succeed or the export fails	Partial files create confusion and break reproducibility.

The preferred design is boring, explicit, and hard to misuse. That is not a limitation. It is the point. Every additional abstraction in the export layer increases the chance that the report looks correct but means something slightly different from what the analysis engine produced. The project cannot afford that drift.

Risks and Failure Modes

- Markdown escaping bugs can corrupt evidence tables or code blocks and make the report unreadable.
- Mermaid syntax can vary between renderers, so the emitted graph source must use conservative syntax and stable identifiers.
- Path handling bugs can accidentally write outside the requested output directory if validation is weak.
- If the renderer starts making content decisions, the export layer will slowly become a second analysis engine. That is the architectural disease to avoid.
- Large reports can produce sizable outputs, so the implementation must avoid unnecessary buffering beyond the single in-memory render pass.

Unsupported inputs: raw traces, partially normalized data, incomplete graph derivations, or free-form text blobs pretending to be report objects. Those belong upstream. This module is not a garbage chute for whatever the caller felt like passing in.

Maintenance risk: if future formats are added casually, the module can become an overstuffed formatting layer with format-specific bugs everywhere. The right answer is to add formats only when there is a concrete downstream user and a stable schema to render.

Testing Strategy

- Unit tests for each renderer, including escaping behavior, list ordering, and section ordering.
- Unit tests for filename planning, output directory validation, and overwrite policies.
- Integration tests that render a representative full report bundle into all supported formats and verify the exact files produced.
- Fixture-driven golden tests for Markdown and JSON content so formatting drift is caught immediately.
- Regression tests for atomic write behavior and cleanup after simulated write failures.

Test fixtures should cover empty sections, long evidence lists, special characters, nested bullets, graph payload presence and absence, and cross-platform line ending normalization. The test suite should compare rendered artifacts byte-for-byte where possible and structurally where byte-for-byte comparison would be too brittle for stable metadata.

Integration testing should use temporary directories only. It should verify that the export result matches the actual files on disk, that the file contents match the expected golden snapshots, and that failed exports leave no partial artifacts behind.

Implementation Checklist

7. Define the export input contract and freeze the list of supported output targets.
8. Implement path validation, output directory creation, and overwrite policy handling.
9. Write the Markdown renderer using the report schema, not string parsing.
10. Write the JSON serializer with stable field ordering and normalized whitespace.
11. Add Mermaid source emission only for already-derived graph artifacts.
12. Implement atomic write helpers and cleanup logic for temporary files.
13. Return a structured ExportResult with artifact metadata and checksums.
14. Create golden fixture tests for every supported format and failure path.
15. Run a render-and-diff pass on representative outputs before release.

Future Expansion

- PDF export for shareable offline reports, using the same structured source bundle and deterministic ordering rules.
- HTML export for local browsing and lightweight navigation.
- Optional image or SVG rendering for graphs once there is a concrete UX need.
- Configurable templates or themes, but only after the MVP proves that the core artifact shapes are stable.
- Batch export and archive packaging for multi-run comparisons.

These expansions are natural descendants of the current module because they still start from the same report bundle and the same formatting contract. They should not force the project to invent new analysis paths or change the canonical report model.

Final Recommendation

The design is MVP-ready if the initial target set stays limited to Markdown, JSON, and Mermaid source, and if PDF remains deferred. That keeps the module clean, deterministic, and testable without introducing a rendering stack that will consume half the project budget in formatting edge cases.

The one simplification still recommended is to keep the export options intentionally small at launch. Resist the urge to add template engines, theming systems, or alternate layout pipelines until there is a real user need. The module should be a reliable file emitter for a structured report, not a miniature publishing platform.